Real-Time Library Seat Occupancy Management System

Project Proposal for UCS503P
Submitted to: Ms. Jhonsy Bansal
Thapar Institute of Engineering and Technology

Saksham Kansal, CSED* Ishaan Gupta, CSED† Sanshi, CSED‡

August, 2026

1 Higher-order goal

This project aims to improve campus smart infrastructure and optimize library resource allocation by providing real-time visibility into seat availability across multi-floor library facilities[cite: 1]. While space utilization challenges affect large educational institutions globally, the immediate engineering objective is to translate IoT-based physical sensing into a low-latency, deployable software and hardware monitoring solution[cite: 1].

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping single-floor IR sensing arrays and validating real-time web state synchronization[cite: 1].
- We will prioritize fast feedback over perfection by using weekly iterations and setting up Continuous Integration (CI) early to reduce release friction[cite: 1].
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback[cite: 1].

3 Problem Statement

Students visiting university libraries face delays and uncertainty when searching for available study desks, particularly during peak exam periods[cite: 1]. Common pain points include:

- **Fragmentation:** seat search relies on physical navigation across multiple floors and aisles, causing noise and disruption[cite: 1].
- **Low transparency:** students cannot easily see the current status of individual seating areas before entering (e.g., "vacant", "occupied")[cite: 1].
- **Slow coordination:** library administrators lack a unified, trackable view of real-time seat utilization across different floors[cite: 1].

*Roll No: 1024031135
†Roll No: 1024031118
‡Roll No: 1024031137

- **Poor data quality:** static or manual entrance headcount boards provide inaccurate, lagging information regarding actual chair availability[cite: 1].

Why this matters now (contextual relevance): In an academic setting with high student density, even small process improvements in seating visibility can noticeably reduce search times and eliminate overcrowded reading hall friction[cite: 1].

4 Proposed Solution

4.1 Overview

Build a web-based and IoT-enabled "Library Seat Management System" with the following components[cite: 1]:

- **Edge sensing hardware:** ESP32 microcontrollers connected to infrared (IR) sensors placed on individual study seats[cite: 1].
- **Real-time state database:** Firebase Realtime Database for instant state synchronization between microcontrollers and web clients[cite: 1].
- **Interactive seat visualization portal:** React web dashboard rendering dynamic top-view maps of each library floor with green (vacant) and red (occupied) visual indicators[cite: 1].
- **Kiosk/Entrance display view:** dynamic floor summaries and availability counters for entrance monitors[cite: 1].

4.2 Core workflow

1. IR sensor detects physical seat presence (student sitting or desk occupied)[cite: 1].
2. ESP32 reads sensor pin state, applies debouncing logic, and transmits updated seat status JSON payload to Firebase Realtime Database via Wi-Fi[cite: 1].
3. Firebase immediately streams database changes to all connected web clients in real-time[cite: 1].
4. React web app dynamically re-renders the visual desk layout (Green $\rightarrow$ Red or Red $\rightarrow$ Green)[cite: 1].
5. Student views real-time floor availability on web app/kiosk and navigates directly to a vacant seat[cite: 1].

4.3 Operational constraints for an educational, tier-2/3 setting

- Keep hardware cost low using economical active IR sensors and multiplexing modules (e.g., 74HC165 / MCP23017) to expand ESP32 pin capacity[cite: 1].
- Avoid dependence on expensive high-end camera systems or research-level AI algorithms; focus on reliable hardware workflow, automation, and system correctness[cite: 1].
- Design lightweight SVG visual floor maps usable on low-to-mid range smartphones and public entrance displays via responsive web design[cite: 1].

5 Solution Approach

This is an engineering course project, so we focus on scalable hardware integration, database state management, and responsive front-end design rather than novel algorithm research[cite: 1].

5.1 Sensor state filtering & quality assistance

Presence detection filtering will be heuristic-based:

- Normalize raw GPIO input signals from IR modules[cite: 1].
- Apply software debouncing (3-second hysteresis window) to prevent rapid flickering from accidental hand movements[cite: 1].
- Send diagnostic flags to staff dashboard if a sensor maintains constant state for anomaly detection (e.g., sensor obstruction); never auto-delete or auto-disable hardware nodes without staff review[cite: 1].

5.2 System architecture

A typical 3-tier IoT web architecture:

- **Frontend:** responsive UI built in React for students and staff (interactive SVG floor maps, layout switcher, summary statistics)[cite: 1].
- **Backend / Real-time Sync API:** Firebase Realtime Database handling authentication, state updates, and security rules[cite: 1].
- **Hardware / Database Node:** ESP32 microcontrollers streaming binary sensor states to cloud paths[cite: 1].

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every repository push[cite: 1].
- Produce repeatable deployment artifacts to static hosting (Firebase Hosting / Vercel)[cite: 1].
- Enable safe and frequent releases of incremental dashboard features[cite: 1].

6 Evaluation Criterion

We define success using measurable metrics tied to the core workflow[cite: 1].

6.1 Primary evaluation metric

End-to-End State Update Latency (E2EL): time between physical seat occupancy event and updated DOM rendering on the React dashboard[cite: 1].

- Target: median E2EL $\leq$ 1.0 second during pilot window[cite: 1].
- Attribution: measured from database timestamps (ESP32 trigger timestamp vs. React render event timestamp)[cite: 1].

6.2 Secondary evaluation metrics

- **Detection Accuracy:** percentage of physical seat states correctly reflected without false positives ($> 98\%$)[cite: 1].
- **Search Time Reduction:** reduction in average time students spend locating a seat compared to baseline manual searching[cite: 1].
- **User clarity:** student-reported clarity of floor map UI using a simple 1–5 feedback scale[cite: 1].
- **Reliability:** availability of cloud sync and dashboard during operational hours ($\geq 99\%$ uptime in pilot)[cite: 1].

6.3 Pilot validation plan

- Recruit 10–20 pilot users (students) and 2–3 library staff roles[cite: 1].
- Compare metrics before/after within the iteration window by logging baseline estimates via short interviews or existing process observations[cite: 1].

7 Scalability

The proposition should scale in both theoretical and practical senses[cite: 1].

1. **Scalable (theoretically):** The system should support growth in number of monitored seats, floors, and concurrent dashboard users by:
   - decoupling hardware nodes, database streams, and React view components,[cite: 1]
   - enabling indexed database paths for floor-level streaming,[cite: 1]
   - using stateless component design.[cite: 1]
2. **Operationally deployable (practically):** The system should run on standard cloud hosting and modular microcontrollers:
   - managed real-time database,[cite: 1]
   - platform-hosted web deployment,[cite: 1]
   - CI/CD pipeline for staging/production.[cite: 1]

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as an IoT web-based project:

- React web framework for UI components and interactive map rendering[cite: 1].
- Managed real-time JSON database (Firebase Realtime Database)[cite: 1].
- ESP32 microcontrollers with standard C++/Arduino Wi-Fi libraries[cite: 1].
- Hosting targets (Firebase Hosting / Vercel) for instant static deployment[cite: 1].
- Test framework for unit/integration testing[cite: 1].
- CI runner and staging deployment target (GitHub Actions)[cite: 1].

We will use these mature components to focus on workflow design, correctness, and measurement rather than inventing new infrastructure[cite: 1].

9 Project scope and deliverables

9.1 Initial deliverable

- 4-seat ESP32 hardware prototype with IR sensors on test rig[cite: 1]
- Firebase Realtime Database schema and sync logic[cite: 1]
- React floor map view (single floor) with dynamic Red/Green indicators[cite: 1]
- Basic logging and timestamps for E2EL measurement[cite: 1]
- CI: build + frontend unit tests + linting[cite: 1]
- CD to staging with a single command or pipeline job[cite: 1]

9.2 Subsequent deliverables

- Multi-floor view navigation and floor switcher[cite: 1]
- I/O expander integration for scaling to 16+ seats per ESP32[cite: 1]
- Admin/ops dashboard for historical occupancy metrics and sensor diagnostics[cite: 1]
- Entrance kiosk display mode with summarized availability metrics[cite: 1]

10 Risks and mitigations

- **Ambient light interference on IR sensors:** use active optical proximity sensors with shielding tubes and software threshold debouncing[cite: 1].
- **Wi-Fi connectivity dropouts:** implement local reconnection logic on ESP32 with state caching upon reconnect[cite: 1].
- **Evaluation measurement ambiguity:** instrument timestamps and define clear status transitions before hardware deployment[cite: 1].
- **Operational issues in pilot:** use staging early; ensure CI/CD produces repeatable deployments[cite: 1].

11 Summary

This project proposes a deployable IoT and web platform to reduce library seat searching delays[cite: 1]. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially End-to-End State Update Latency)[cite: 1]. It remains focused on engineering workflow and scalability readiness rather than research-driven novelty[cite: 1].